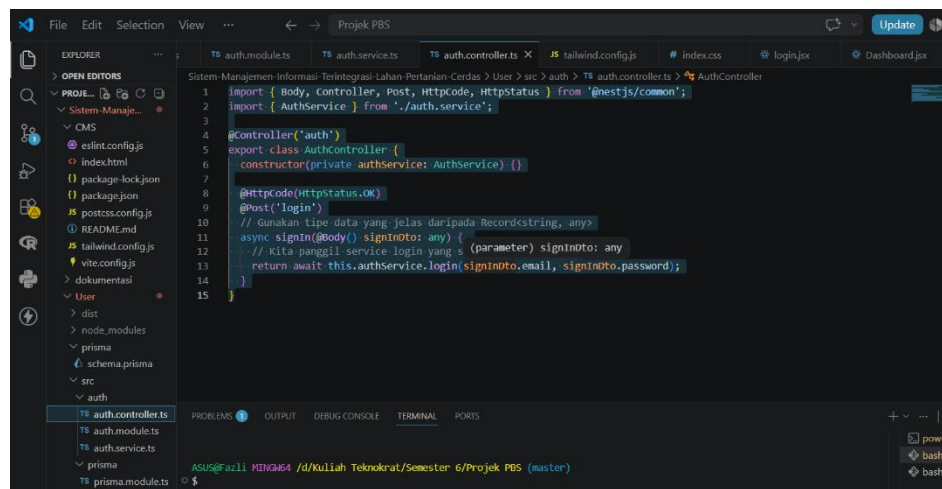


Nama : M Fazli Anan

NPM 23312072

1. Gerbang otentikasi administrator pada sistem dikendalikan penuh oleh layanan backend melalui pemanggilan route POST /auth/login. Proses pengujian dilakukan dengan mengirimkan objek data berupa alamat surel dan sandi akun dalam format JSON, di mana modul validasi server NestJS akan mengecek kesesuaian kredensial tersebut dengan rekaman data pengelola di dalam database PostgreSQL. Apabila data terverifikasi sah, server memancarkan respon balik dengan status HTTP 200 OK yang mengangkut kode akses unik access_token berbasis JWT serta lampiran data profil user_data untuk kemudian disimpan pada penyimpanan lokal browser oleh Vivi sebagai pengembang sisi frontend.



```
File Edit Selection View ... Projek PBS
EXPLORER
  PROJE...
  Sistem Manaje...
    CMS
      eslint.config.js
      index.html
      package-lock.json
      package.json
      postcss.config.js
      README.md
      tailwind.config.js
      vite.config.js
      dokumentasi
    User
      dist
      node_modules
      prisma
        schema.prisma
      src
        auth
          TS auth.controller.ts
          TS auth.module.ts
          TS auth.service.ts
        prisma
          TS prisma.module.ts

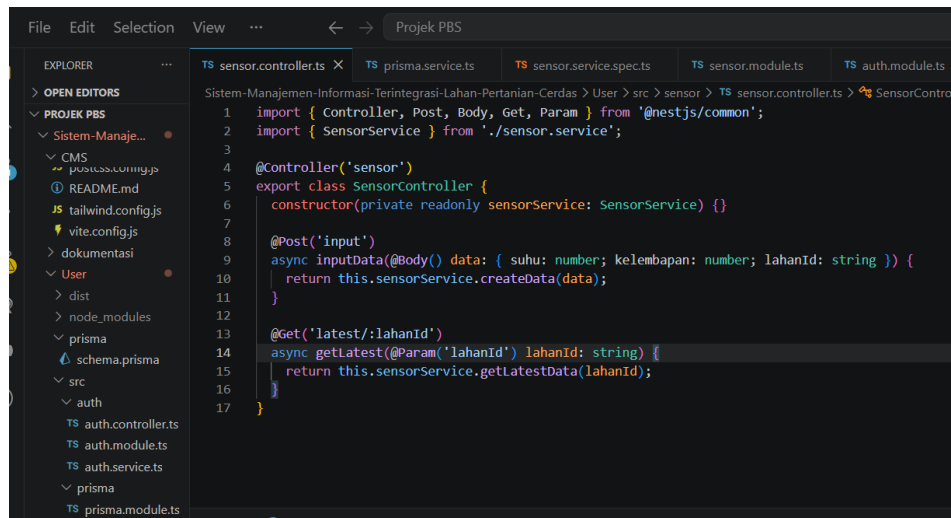
TS auth.controller.ts
1 import { Body, Controller, Post, HttpCode, HttpStatus } from '@nestjs/common';
2 import { AuthService } from './auth.service';
3
4 @Controller('auth')
5 export class AuthController {
6   constructor(private authService: AuthService) {}
7
8   @HttpCode(HttpStatus.OK)
9   @Post('login')
10  // Gunakan tipe data yang jelas daripada Record<string, any>
11  async signIn(@Body() signInDto: any) {
12    // Kita panggil service login yang s (parameter) signInDto: any
13    return await this.authService.login(signInDto.email, signInDto.password);
14  }
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

ASUS@Fazli MINGM4 /d/Kuliah Teknokrat/Semester 6/Projek PBS (master)

\$

2. Mekanisme penarikan parameter indikator makro area perkebunan secara berkala diakomodasi oleh service pada endpoint GET /sensor. Ketika request dikirimkan ke endpoint ini, server akan memproses kueri data guna menyajikan respon berstatus 200 OK yang memuat paket informasi dinamis *real-time* mengenai tingkat kelembapan tanah, suhu udara, kecukupan nutrisi unsur NPK, hingga visualisasi fase siklus vegetatif dari tanaman. Aliran data terpusat inilah yang memegang peran krusial sebagai penyuplai pasokan informasi bagi komponen antarmuka visual berupa blok-blok perkebunan seperti kopi robusta, kelapa sawit, maupun karet pada aplikasi utama.



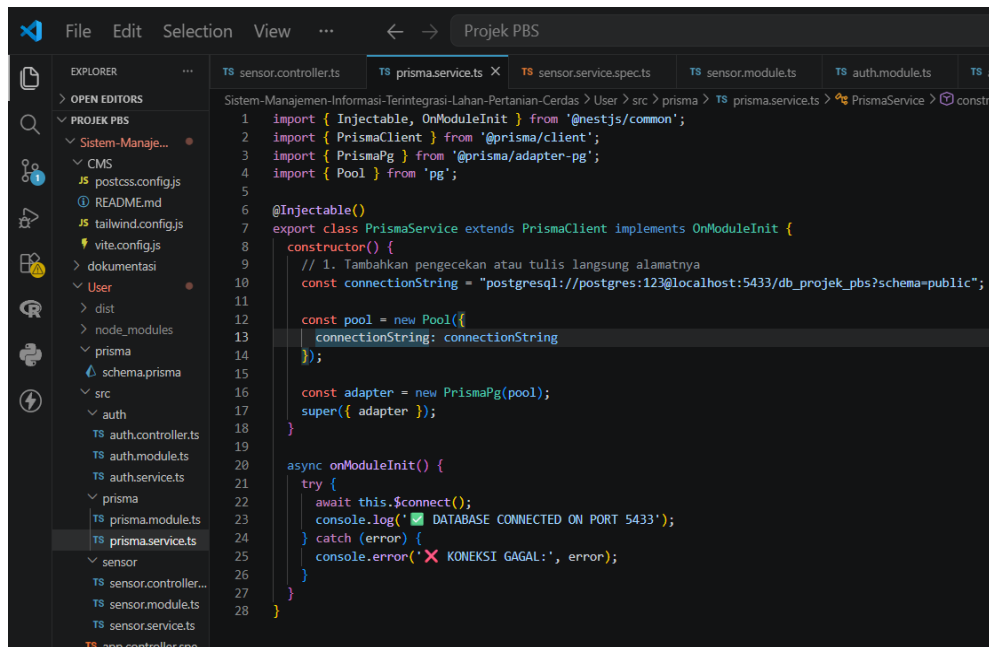
```
File Edit Selection View ... < -> Projek PBS

EXPLORER
  > OPEN EDITORS
  > PROJEK PBS
    > Sistem-Manaje...
      > CMS
        postcss.config.js
      > README.md
      > JS
        tailwind.config.js
        vite.config.js
      > dokumentasi
      > dist
      > node_modules
      > prisma
        schema.prisma
      > src
        > auth
          TS auth.controller.ts
          TS auth.module.ts
          TS auth.service.ts
        > prisma
          TS prisma.module.ts

TS sensor.controller.ts X TS prisma.service.ts TS sensor.service.spec.ts TS sensor.module.ts TS auth.module.ts

1 import { Controller, Post, Body, Get, Param } from '@nestjs/common';
2 import { SensorService } from './sensor.service';
3
4 @Controller('sensor')
5 export class SensorController {
6   constructor(private readonly sensorService: SensorService) {}
7
8   @Post('input')
9   async inputData(@Body() data: { suhu: number; kelembapan: number; lahanId: string }) {
10     return this.sensorService.createData(data);
11   }
12
13   @Get('latest/:lahanId')
14   async getLatest(@Param('lahanId') lahanId: string) {
15     return this.sensorService.getLatestData(lahanId);
16   }
17 }
```

- Arsitektur data layer dan stabilitas pipa komunikasi server aplikasi dioptimalkan menggunakan Prisma ORM dengan konfigurasi port database lokal 5433. Dalam mengatasi restriksi enkapsulasi keamanan bawaan dari PostgreSQL yang sering memicu kendala otentikasi ketat (*SASL error*), sistem dikonfigurasi menggunakan tautan *connection pooler* mandiri dari library pg (PrismaPg) yang disuntikkan secara eksplisit pada initialization service utama. Implementasi ini menjamin pertukaran data antara logika bisnis NestJS dan pangkalan data relasional berjalan secara *persistent*, efisien, serta sanggup mengantisipasi lonjakan beban trafik dari penyiaran data sensor perkebunan skala luas.



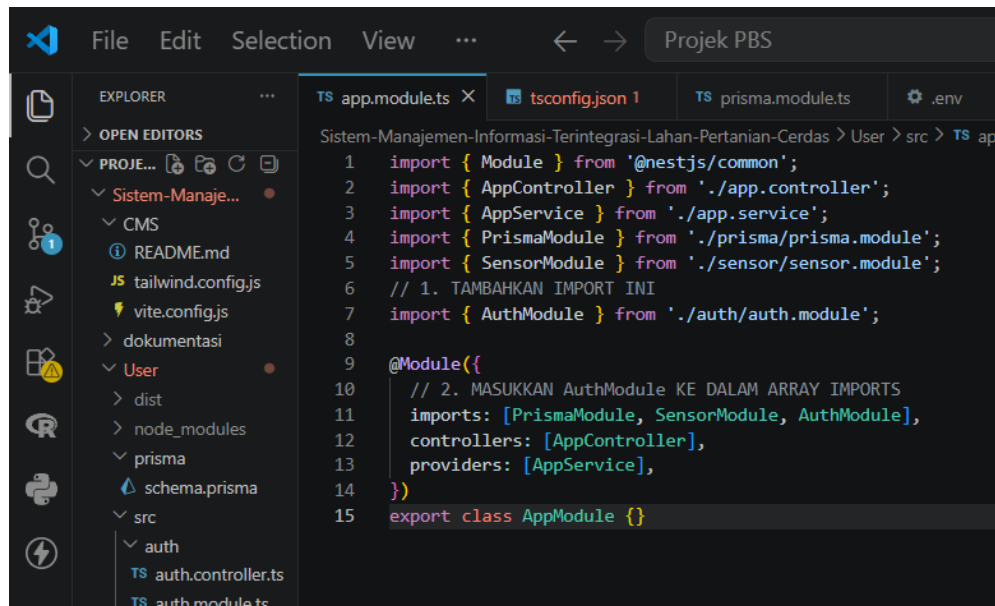
```
File Edit Selection View ... < -> Projek PBS

EXPLORER
  > OPEN EDITORS
  > PROJEK PBS
    > Sistem-Manaje...
      > CMS
        postcss.config.js
      > README.md
      > JS
        tailwind.config.js
        vite.config.js
      > dokumentasi
      > dist
      > node_modules
      > prisma
        schema.prisma
      > src
        > auth
          TS auth.controller.ts
          TS auth.module.ts
          TS auth.service.ts
        > prisma
          TS prisma.module.ts
          TS prisma.service.ts
        > sensor
          TS sensor.controller...
          TS sensor.module.ts
          TS sensor.service.ts
        > app.controller.spe...

TS sensor.controller.ts TS prisma.service.ts X TS sensor.service.spec.ts TS sensor.module.ts TS auth.module.ts TS a...

1 import { Injectable, OnModuleInit } from '@nestjs/common';
2 import { PrismaClient } from '@prisma/client';
3 import { PrismaPg } from '@prisma/adapter-pg';
4 import { Pool } from 'pg';
5
6 @Injectable()
7 export class PrismaService extends PrismaClient implements OnModuleInit {
8   constructor() {
9     // 1. Tambahkan pengecekan atau tulis langsung alamatnya
10    const connectionString = "postgresql://postgres:123@localhost:5433/db_projek_pbs?schema=public";
11
12    const pool = new Pool({
13      connectionString: connectionString
14    });
15
16    const adapter = new PrismaPg(pool);
17    super({ adapter });
18   }
19
20   async onModuleInit() {
21     try {
22       await this.$connect();
23       console.log('✅ DATABASE CONNECTED ON PORT 5433');
24     } catch (error) {
25       console.error('❌ KONEKSI GAGAL:', error);
26     }
27   }
28 }
```

- Integrasi dan koordinasi seluruh komponen logika bisnis dikelola secara terpusat melalui konfigurasi modul utama aplikasi (*AppModule*). Berkas root module ini berfungsi sebagai dirigen utama sistem NestJS yang mendaftarkan sekaligus menggabungkan *AuthModule* (untuk gerbang login), *SensorModule* (untuk penarikan data blok perkebunan), serta *PrismaModule* yang mengasuh koneksi database PostgreSQL. Pengelompokan berbasis arsitektur *module-driven* ini memastikan seluruh layanan *controller* dan *provider* dapat saling mengenali dependensi satu sama lain secara efisien, terstruktur, dan siap dieksekusi saat server backend TerraVision mulai dinyalakan.



```
File Edit Selection View ... < > Projek PBS

EXPLORER
> OPEN EDITORS
PROJE...
  Sistem-Manaje...
    CMS
      README.md
      tailwind.config.js
      vite.config.js
      dokumentasi
    User
      dist
      node_modules
      prisma
      schema.prisma
      src
        auth
          TS auth.controller.ts
          TS auth.module.ts

TS app.module.ts x tsconfig.json 1 TS prisma.module.ts .env

Sistem-Manajemen-Informasi-Terintegrasi-Lahan-Pertanian-Cerdas > User > src > TS ap
1 import { Module } from '@nestjs/common';
2 import { AppController } from './app.controller';
3 import { AppService } from './app.service';
4 import { PrismaModule } from './prisma/prisma.module';
5 import { SensorModule } from './sensor/sensor.module';
6 // 1. TAMBAHKAN IMPORT INI
7 import { AuthModule } from './auth/auth.module';
8
9 @Module({
10   // 2. MASUKKAN AuthModule KE DALAM ARRAY IMPORTS
11   imports: [PrismaModule, SensorModule, AuthModule],
12   controllers: [AppController],
13   providers: [AppService],
14 })
15 export class AppModule {}
```